

CESE5040

High-Performance and AI Architectures

2026 Q4

Student Information

Name: Daniel Tyukov

Student Number: 5714699

Lab: Lab Assignment 4

Notes

- Hardware: Intel Core Ultra 9 185H, CPU only. JAX 0.10.0, Flax 0.12.7, Optax 0.2.8, Python 3.12.
- Submitted: L4_Q4.py and L4_Q6.py. Q4.5 uses the supplied `m1p_adv.py` architecture unchanged.
- Each accuracy is one run at the stated seed. Wall times cover the training loop only.

Lab Exercise 4

Question 4.1 - Backprop by hand and automatic differentiation

1. Let $z_1^1 = w_{11}^1 x_1 + w_{21}^1 x_2 + b_1^1$, $z_2^1 = w_{12}^1 x_1 + w_{22}^1 x_2 + b_2^1$, and $z^2 = w_{11}^2 h_1 + w_{21}^2 h_2 + b_1^2$, with $h_i = \text{ReLU}(z_i^1)$ and $y = \sigma(z^2)$. The loss is $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$, so $\partial \mathcal{L} / \partial y = (y - \hat{y})$ and $\partial y / \partial z^2 = y(1 - y)$ from the sigmoid derivative.

For w_{11}^2 the chain runs $\mathcal{L} \rightarrow y \rightarrow z^2 \rightarrow w_{11}^2$:

$$\frac{\partial \mathcal{L}}{\partial w_{11}^2} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial z^2} \frac{\partial z^2}{\partial w_{11}^2} = (y - \hat{y}) y(1 - y) h_1.$$

For w_{21}^1 the chain goes one layer further back, through h_1 :

$$\frac{\partial \mathcal{L}}{\partial w_{21}^1} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial z^2} \frac{\partial z^2}{\partial h_1} \frac{\partial h_1}{\partial z_1^1} \frac{\partial z_1^1}{\partial w_{21}^1} = (y - \hat{y}) y(1 - y) w_{11}^2 \mathbb{K}[z_1^1 > 0] x_2.$$

The ReLU derivative is the indicator $\mathbb{K}[z_1^1 > 0]$, which is why dead ReLU units block any gradient signal from reaching w_{21}^1 .

2. Autodiff frameworks record every elementary op of the forward pass along with its local Jacobian, then walk that graph to compute exact derivatives. Forward mode is cheap when there are few inputs and many outputs; reverse mode (backprop) is cheap in the opposite case, which is what training a scalar loss against millions of parameters demands. JAX, PyTorch, and TensorFlow trace the user's Python into an intermediate representation and apply the chain rule along the recorded edges. The user only writes the forward pass; gradients come back exactly and at near-forward cost, which is what makes deep networks trainable at all.

Question 4.2 - MLP parameter count and CNN motivation

1. With I input neurons, L hidden layers of width H , and O outputs, the layer-by-layer count is:

- Input to first hidden: $IH + H$ (weights plus bias).
- Between consecutive hidden layers: $H^2 + H$, repeated $L-1$ times.
- Last hidden to output: $HO + O$.

$$\text{\#params} = IH + H + (L-1)(H^2 + H) + HO + O.$$

For an MNIST-sized example with $I = 784$, $L = 2$, $H = 512$, $O = 10$ this gives about 670k parameters, which is already large for a simple toy.

2. Scaling this up is hopeless. A single $224 \times 224 \times 3$ ImageNet image flattened into a $H = 4096$ hidden layer already needs $\approx 617\text{M}$ parameters in one weight matrix. CNNs fix this with two priors: locality (each filter looks at a $k \times k$ window) and translation invariance (the same filter slides over the whole image). The parameter count drops from $H_{\text{in}} \cdot H_{\text{out}}$ to $k \cdot k \cdot C_{\text{in}} \cdot C_{\text{out}}$, which is independent of input resolution and several orders of magnitude smaller for typical $k = 3$ or 5 .

Question 4.3 - Batching, learning rate, fit quality, loss vs accuracy

1. Two reasons. Averaging the loss over a batch lowers the variance of the gradient estimate, so each step points more reliably than a single sample would. And hardware: GPUs need wide tensors to fill their tensor units, so a batch of 128 runs almost as fast as one image but does 128 times the useful work. A subtler effect is that very large batches lose the regularising noise of stochastic updates and settle into sharper minima that generalise worse. How the data is split also matters. A fixed order lets the model exploit it, so the convention is to reshuffle every epoch; with imbalanced classes a single-class batch will bias the gradient.

2. Too large and updates overshoot the minimum at every step; loss oscillates or diverges, often NaN within a few steps. Too small and the model moves in the right direction but never reaches a useful minimum in any reasonable budget. A scheduled rate gets both: start large to escape the random init, then decay (step, cosine, one-cycle) so the optimiser can settle. A short warmup at the very start helps when the early gradients are poorly conditioned.

3. Overfitting: training accuracy is high while test accuracy stalls or drops. The signature is a widening train-test gap (training loss still falling while validation loss climbs). Underfitting is the opposite: both train and test accuracy stay low because the model is too small or trained too short. Fixes are regularisation (L2, dropout), early stopping, or more data on the overfit side, and more capacity or more epochs on the underfit side.

4. The loss is a continuous, differentiable proxy for what we actually care about (top-1 accuracy here). It tracks the probability assigned to the correct class, so it reacts to the margin of every prediction; accuracy is binary per example. A model can keep driving cross-entropy down by becoming overconfident on examples it already gets right. That does nothing for accuracy and tends to coincide with overfitting:

training loss keeps dropping while test accuracy plateaus. That is why the early-stopping signal of choice is validation accuracy or validation loss, not training loss.

Question 4.4 - Simple MLP on MNIST (L4_Q4.py)

1. Trained vs untrained accuracy at three seeds.

Seed	Untrained acc (%)	Train acc (%)	Test acc (%)
0	10.06	97.71	97.08
10	9.41	97.74	97.10
42	5.84	97.87	97.25

Table 1: Per-sample SGD, $\alpha = 10^{-3}$, 10 epochs. Test accuracy spans only 0.17 pp across seeds.

Untrained accuracy sits at chance ($\approx 10\%$ for ten classes), with seed 42 unlucky on the test set by random init. After ten epochs all three seeds land within 0.2 pp of each other. The seed moves the trajectory but not the final operating point, so for this small model trained accuracy is essentially seed-independent.

2. Effect of removing the $[0,1]$ normalization.

Dropping the $x/255$ step at seed 10 collapsed test accuracy from 97.10% to **11.34%** (train 11.24%), barely above chance. Raw pixels are about two orders of magnitude larger than the default LeCun-normal init expects, so the first pre-activations are huge, the gradient saturates, and the very first SGD step blows the parameters past any usable region. Normalising keeps activation statistics in the range the init was designed for.

3. Learning-rate sweep at $\alpha \in \{10^{-1}, 10^{-4}, 10^{-7}\}$, seed 10.

α	Train acc (%)	Test acc (%)
10^{-1}	81.60	82.13
10^{-3} (baseline)	97.74	97.10
10^{-4}	92.14	92.55
10^{-7}	11.39	11.17

Table 2: Per-sample SGD, 10 epochs. The 10^{-3} row is reused from 4.4.1 for context.

At $\alpha = 0.1$ updates are too aggressive for per-sample SGD; the model picks up the easy digits but never settles, ending 15 pp below baseline. At $\alpha = 10^{-4}$ the trajectory is healthy but ten epochs are not enough. At $\alpha = 10^{-7}$ the steps are so tiny that the parameters barely move from their random init.

4. Mini-batching for batch sizes 32, 64, 128, 256, 512.

Wall time drops because each batch is one JAX dispatch doing more work, and accuracy drops too because the number of weight updates per epoch shrinks from 60,000 (per-sample) to 117 (batch 512). With α pinned at 10^{-3} those few updates are not enough. This is the textbook setup for the linear scaling rule of learning rate to batch size, which 4.4.5 verifies.

5. Increasing the learning rate for batch size 64.

The same model that capped at 87% with $\alpha = 10^{-3}$ reaches 97.5% at $\alpha = 10^{-1}$, matching the per-sample

Batch	Train acc (%)	Test acc (%)	Wall (s)
32	89.39	89.93	4.5
64	87.02	87.75	3.6
128	83.34	84.28	2.4
256	77.22	78.18	1.8
512	67.84	68.43	1.4

Table 3: $\alpha = 10^{-3}$, 10 epochs, seed 10. Compare against the per-sample baseline at the same α (97.10% test, 85 s wall).

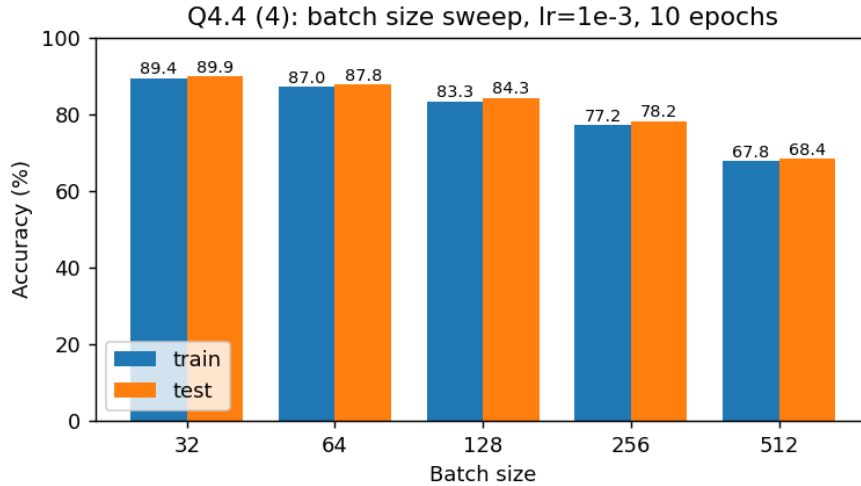


Figure 1: Test accuracy falls monotonically as batch size grows, at fixed α .

baseline from 4.4.1. A batch gradient is the average of 64 sample gradients, so the per-step magnitude is roughly $1/64$ of the per-sample case and the rate has to be scaled up by a comparable factor. Beyond $\alpha \approx 0.1$ further gains shrink (train climbs, test stays put), which is the first whiff of overfitting.

6. Training set reduced to 1000 samples, 20 epochs, batch 64.

With $\alpha = 10^{-3}$ at batch 64 there are only $1000/64 \cdot 20 \approx 312$ updates total, not enough to fit 1000 samples with a 101k-parameter model, so both numbers stay at the underfit floor. Switching to the working $\alpha = 10^{-1}$ from 4.4.5 fits the training set almost perfectly (96.1%) while test accuracy lags ten points behind. That is the textbook overfit regime: plenty of capacity, too little data, no regularisation. On the full 60k set the same network generalises fine because more data acts as the regulariser.

Question 4.5 - Regularised MLP (mlp_adv.py)

1. *What is dropout?* Dropout randomly zeroes a fraction p of the activations during each training step and rescales the survivors by $1/(1 - p)$ so the expected magnitude is preserved. At inference it is off and every unit participates. The effect is two-fold: neurons cannot co-adapt because any of them might be dropped, and the procedure is an implicit ensemble over the 2^n subnetworks defined by the dropout masks.

2. *Is deeper / wider always better?* No. More layers and more width add capacity but also add param-

α	Train acc (%)	Test acc (%)
10^{-3} (baseline)	87.02	87.75
10^{-2}	93.37	93.40
10^{-1}	98.40	97.51
0.5	99.35	97.65

Table 4: Batch 64, seed 10, 10 epochs.

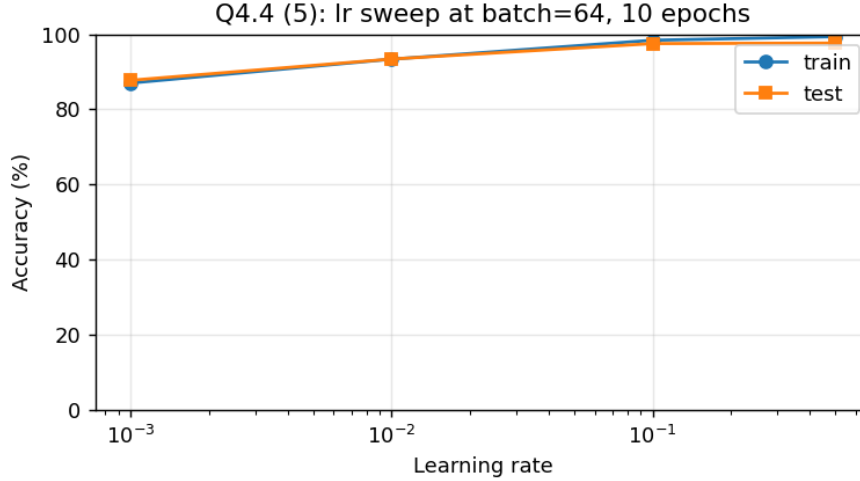


Figure 2: Bumping the learning rate recovers the accuracy lost by batching, exactly as the linear scaling rule predicts.

ters, memory, training time, and the risk of overfitting plus vanishing or exploding gradients (especially without residual connections or careful init). A 512-512 MLP already saturates MNIST; a 4096-4096-4096 stack would learn nothing the smaller model misses and just train slower. Depth pays off when the data has compositional structure, which is more true of images, audio and language than of MNIST digits.

3. AdamW and weight decay. Adam combines momentum (first moment of the gradient) with per-parameter step normalisation (second moment), so each parameter takes a step adapted to its own gradient scale. AdamW is a small fix on top. Vanilla Adam folds an L2 penalty into the gradient before the moment normalisation, which ties the effective decay to the gradient magnitude and breaks the regularisation. AdamW decouples the decay and applies it directly to the parameters at update time, so the decay coefficient behaves as a clean shrinkage term. Its role matches L2: pull weights toward zero, keep them small, discourage overfitting.

4. Three-seed accuracy on `m1p_adv.py`, 10 epochs.

Test accuracy of 97.96-98.24% across seeds sits about a percentage point above the plain `m1p.py` baseline (97.08-97.25%), with a train-test gap of about 1.3 pp. Three changes pull the same way: wider 512-512 hidden layers fit better than 128 units, AdamW converges faster than vanilla SGD because each parameter’s step is rescaled by its own gradient variance, and dropout keeps the extra capacity from turning into extra overfitting.

α	Train acc (%)	Test acc (%)	Gap (pp)
10^{-3}	29.10	26.09	3.0
10^{-1}	96.10	86.62	9.5

Table 5: 1000 samples, 20 epochs, batch 64, seed 10. Two regimes shown.

Seed	Untrained acc (%)	Train acc (%)	Test acc (%)
0	6.65	99.40	97.96
10	8.78	99.38	98.12
42	9.47	99.56	98.24

Table 6: `m1p_adv.py` architecture (Dense 512, Dropout 0.3, Dense 512, Dropout 0.3, Dense 10), AdamW with $\alpha = 10^{-3}$, weight decay 10^{-4} , batch 128, 10 epochs. Roughly 670,000 trainable parameters.

Question 4.6 - LeNet-5 (`L4_Q6.py`)

1. *Parameter count.* The LeNet-5 architecture from the assignment figure has the following parameter breakdown:

Layer	Params	Calculation
C1 (Conv $1 \rightarrow 6$, 5×5)	156	$5 \cdot 5 \cdot 1 \cdot 6 + 6$
S2 (AvgPool 2×2)	0	no parameters
C3 (Conv $6 \rightarrow 16$, 5×5)	2,416	$5 \cdot 5 \cdot 6 \cdot 16 + 16$
S4 (AvgPool 2×2)	0	no parameters
F5 (Dense $400 \rightarrow 120$)	48,120	$400 \cdot 120 + 120$
F6 (Dense $120 \rightarrow 84$)	10,164	$120 \cdot 84 + 84$
Out (Dense $84 \rightarrow 10$)	850	$84 \cdot 10 + 10$
Total	61,706	

Table 7: LeNet-5 parameter breakdown. Almost 80% of the parameters live in F5 alone.

The two conv stages together hold only ≈ 2.6 k parameters even though they do most of the feature extraction; the dense layers carry the rest. That is already an order of magnitude smaller than the 670k of a 512-512 MLP.

2. *Implementation notes.* Built with `flax.linen`. C1 uses SAME padding so the first feature map stays 28×28 (the original LeNet padded the input to 32×32 ; using SAME on the conv is the modern equivalent). C3 uses VALID padding to get 10×10 . Both pools are 2×2 stride 2. All conv and dense layers use `tanh`. Optimiser is AdamW, batch 128.

3. *Three-seed run at the default hyper-parameters, then a 3×3 LR / weight-decay sweep at seed 10. All runs 10 epochs, batch 128.*

Learning rate carries almost all the signal. $\alpha = 10^{-3}$ is the sweet spot at 98.27%, $\alpha = 10^{-2}$ overshoots (the optimiser oscillates near the minimum), $\alpha = 10^{-4}$ undertrains in 10 epochs. Weight decay barely moves the needle at $\alpha = 10^{-3}$ and is invisible at $\alpha = 10^{-4}$, because AdamW's per-step shrinkage is proportional to $\alpha \cdot \lambda_{\text{wd}}$, so a small rate shrinks the decay scale too. The one place decay pays off is at $\alpha = 10^{-2}$, where stronger regularisation damps the oscillation and recovers about 0.8 pp.

4. *LeNet-5 vs MLP comparison.*

Seed	Untrained acc (%)	Train acc (%)	Test acc (%)
0	6.45	99.32	98.40
10	9.88	99.37	98.27
42	7.89	99.37	98.48

Table 8: LeNet-5, AdamW with $\alpha = 10^{-3}$, $\lambda_{\text{wd}} = 10^{-4}$. Test accuracy spans 0.21 pp across seeds.

α	$\lambda_{\text{wd}} = 0$	$\lambda_{\text{wd}} = 10^{-4}$	$\lambda_{\text{wd}} = 10^{-3}$
10^{-2}	96.49	96.71	97.31
10^{-3}	98.27	98.27	98.25
10^{-4}	96.75	96.75	96.75

Table 9: Test accuracy from the LR $\times$ weight-decay sweep, seed 10, 10 epochs.

LeNet-5 has roughly ten times fewer parameters than the wide MLP (61.7k vs 670k) and still generalises a few tenths of a percentage point better. The reason is inductive bias. A conv layer encodes two priors: pixels close together are more related than pixels far apart, and the same feature can appear anywhere in the image. Both hold for digits, so the parameter budget goes into filters that match the input rather than into independent pixel-to-hidden-unit edges. The MLP has neither prior; most of its 670k parameters re-discover correlations a convolution captures for free. On MNIST the wastage costs only a few tenths of a percentage point. On richer data like CIFAR-10 or ImageNet the same inductive bias is the difference between a model that trains and one that does not.

LLM Usage Disclosure

I used Claude Code on this lab for:

- Cross-checking the Flax conv / pool API and the AdamW signature in Optax against the documentation while implementing LeNet-5.
- Editing the report (tightening phrasing, trimming filler).

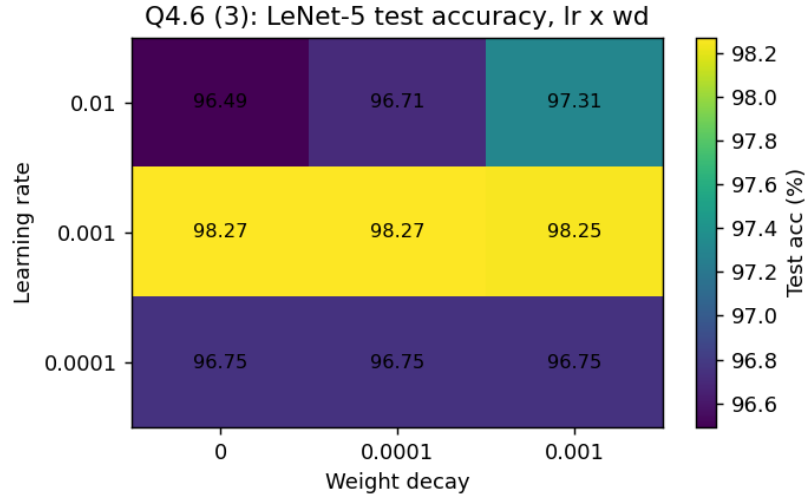


Figure 3: LeNet-5 test accuracy as a function of learning rate and weight decay. The $\alpha = 10^{-3}$ row is uniformly the best regardless of weight decay.

Model	Params	Best test acc (%)	Configuration
mlp.py (per-sample SGD)	$\approx 101k$	97.25	seed 42, $\alpha = 10^{-3}$, 10 epochs
mlp.py (batched, scaled LR)	$\approx 101k$	97.65	seed 10, $\alpha = 0.5$, batch 64
mlp_adv.py (Dropout + AdamW)	$\approx 670k$	98.24	seed 42, $\alpha = 10^{-3}$, $\lambda_{wd} = 10^{-4}$
LeNet-5	61.7k	98.48	seed 42, same hyper-params as above

Table 10: Best test accuracy reached by each model on this lab. LeNet-5 wins despite having an order of magnitude fewer parameters than the deepest MLP.